🧺 CleanFlow

Laundry Management System

Complete Documentation

Product Requirements Document (PRD)

Technical Requirements Document (TRD)

UI/UX Design Documentation

Version: 1.0

Date: July 7, 2026

Reference: WashingBasket.in

Technology Stack: React + Antigravity + Node.js

📑 Table of Contents

1. Executive Summary

> **Project Vision:** To create a seamless, automated laundry management system that connects customers with service providers through real-time WhatsApp notifications, easy scheduling, and transparent pricing.

1.1 Problem Statement

Traditional laundry businesses face challenges in managing customer orders, tracking pickups/deliveries, and maintaining communication. Customers lack visibility into order status and pricing transparency.

1.2 Solution Overview

CleanFlow provides a comprehensive web platform where:

- **Customers** can register via phone/email, schedule pickups, track orders in real-time, and receive WhatsApp updates with QR codes
- **Admins** can manage orders with one-click pickup/delivery confirmations, update pricing, and view analytics
- **WhatsApp Integration** ensures instant communication for order confirmations, pickup alerts, and delivery notifications

1.3 Key Differentiators

> - ✅ WhatsApp-first communication approach
> - ✅ QR code-based order tracking
> - ✅ One-click admin operations (pickup/delivery)
> - ✅ Real-time weight and price calculations
> - ✅ Mobile-responsive design using Antigravity

2. Product Requirements Document (PRD)

2.1 Product Overview

Attribute	Details
Product Name	CleanFlow - Laundry Management System
Reference Website	https://washingbasket.in/user/dashboard/index
Target Platform	Responsive Web Application (Desktop + Mobile)
Primary Users	Customers, Admin/Staff
Development Approach	React.js with Antigravity Design System

2.2 Target Users & Personas

Persona 1: Busy Professional (Rahul, 32)

> **Profile:** IT professional working 9-7, lives alone in metro city
>
> **Needs:**
>
> - Quick pickup scheduling without phone calls
> - Real-time order status tracking
> - WhatsApp notifications for all updates
> - Digital payment options
> - Evening/weekend pickup slots
>
> **Pain Points:** No time for laundry, forgets to schedule, wants hassle-free service

Persona 2: Homemaker (Priya, 42)

> **Profile:** Manages household of 4, regular laundry needs
>
> **Needs:**

- Bulk order placement for family laundry
- Clear price comparison between services
- Weight-based pricing transparency
- WhatsApp confirmation with item details
- Multiple address management

Pain Points: Wants fair pricing, needs reliable pickup/delivery

Persona 3: Laundry Business Owner (Amit, 38)

Profile: Runs a laundry chain with 3 outlets

Needs:

- Centralized order management dashboard
- Automated customer communication
- Staff assignment and tracking
- Revenue analytics and reporting
- Quick pickup/delivery confirmation buttons

Pain Points: Manual coordination with customers, missed deliveries, staff management

2.3 Functional Requirements

2.3.1 Customer Features

Feature ID	Feature Name	Description	Priority
AUTH-01	Phone/Email Registration	Register using mobile number or email with password	P0
AUTH-02	OTP Verification	Verify account via WhatsApp/SMS OTP (6-digit)	P0
AUTH-03	Profile Management	Edit name, email, phone, address details	P0
AUTH-04	Multiple Addresses	Save and manage multiple pickup addresses	P1

ORD-01	Schedule Pickup	Select date, time slot, and address for pickup	P0
ORD-02	Item Selection	Choose clothes type, quantity, and service type	P0
ORD-03	Price Calculator	Real-time price estimation based on items and weight	P0
ORD-04	Order Tracking	Visual timeline showing order status progress	P0
ORD-05	QR Code Receipt	Generate QR code with order details	P0
ORD-06	Order History	View past orders with reorder functionality	P1
ORD-07	Cancel Order	Cancel order within allowed timeframe (before pickup)	P1
COM-01	WhatsApp Updates	All status updates via WhatsApp messages	P0
COM-02	Pickup Confirmation	WhatsApp message when admin confirms pickup	P0
COM-03	Delivery Confirmation	WhatsApp message when admin confirms delivery	P0

2.3.2 Admin Features

Feature ID	Feature Name	Description	Priority
ADM-01	Real-time Dashboard	Overview of all orders with status filters	P0
ADM-02	Pickup Button	One-click pickup confirmation → WhatsApp to customer	P0
ADM-03	Delivery Button	One-click delivery confirmation → WhatsApp to customer	P0
ADM-04	Price Management	Update service prices for different items	P0
ADM-05	Order Management	View, filter, and manage all orders	P0
ADM-06	Send WhatsApp Updates	Manually trigger WhatsApp notifications	P0
ADM-07	Revenue Analytics	Daily/weekly/monthly revenue charts	P1
ADM-08	Customer Management	View customer list and order history	P1

2.4 Non-Functional Requirements

Category	Requirement	Specification
Performance	Page Load Time	< 3 seconds on 4G connection
Performance	API Response Time	< 500ms for 95% of requests
Availability	System Uptime	99.9% (excluding planned maintenance)
Security	Data Encryption	AES-256 for sensitive data at rest
Security	Authentication	JWT with 30-day validity + refresh tokens
Security	Password Policy	Minimum 8 characters, alphanumeric
Scalability	Concurrent Users	Support 10,000+ concurrent users
Usability	Mobile Responsive	100% mobile-compatible across devices
Compliance	Data Privacy	GDPR & Indian IT Act compliant

3. Technical Requirements Document (TRD)

3.1 Technology Stack

Frontend (React + Antigravity)

```
{
  "framework": "React 18.2+",
  "ui_library": "Ant Design (Antd) 5.12+",
  "design_system": "Antigravity Design System",
  "state_management": "Redux Toolkit 1.9+",
  "styling": "TailwindCSS 3.4+ with Ant Design tokens",
  "routing": "React Router v6.20+",
  "http_client": "Axios 1.6+",
  "form_handling": "Formik 2.4+ + Yup 1.3+",
  "icons": "@ant-design/icons 5.2+",
  "charts": "Recharts 2.10+",
  "qr_code": "qrcode.react 3.1+",
  "date_handling": "dayjs 1.11+",
  "notifications": "react-hot-toast 2.4+",
  "build_tool": "Vite 5.0+"
}
```

Backend (Node.js + Express)

```
{
  "runtime": "Node.js 18 LTS",
  "framework": "Express.js 4.18+",
  "database": "MongoDB 7.0 (Atlas)",
  "odm": "Mongoose 8.0+",
  "authentication": "jsonwebtoken 9.0+ + bcryptjs 2.4+",
  "whatsapp_api": "Twilio SDK 4.22+ / WhatsApp Business API v2.49",
  "file_storage": "AWS S3 / Cloudinary",
  "email_service": "Nodemailer 6.9+",
  "scheduling": "node-cron 3.0+",
  "logging": "Winston 3.11+",
  "validation": "Joi 17.11+ / express-validator 7.0+",
  "caching": "Redis 7.0+ (optional)",
```

```
  "queue": "Bull 4.12+ (for WhatsApp messages)"
}
```

Infrastructure & DevOps

```
{
  "hosting": "AWS EC2 t3.medium / Vercel Pro",
  "cdn": "Cloudflare CDN",
  "domain": "cleanflow.app (custom domain)",
  "ssl": "Let's Encrypt / AWS Certificate Manager",
  "monitoring": "Sentry for errors, Datadog for performance",
  "backup": "MongoDB Atlas automated daily backups",
  "ci_cd": "GitHub Actions / GitLab CI",
  "containerization": "Docker + Docker Compose (optional)",
  "environment_variables": ".env with dotenv-safe"
}
```

3.2 Database Schema

Users Collection

```
{
  _id: ObjectId,
  name: { type: String, required: true },
  email: { type: String, required: true, unique: true, lowercase: true },
  phone: { type: String, required: true, unique: true },
  password: { type: String, required: true, minlength: 6 },
  role: {
    type: String,
    enum: ['customer', 'admin', 'staff'],
    default: 'customer'
  },
  addresses: [{
    _id: ObjectId,
    type: { type: String, enum: ['home', 'work', 'other'] },
    label: String,
    street: { type: String, required: true },
    landmark: String,
    city: { type: String, required: true },
    state: String,
    pincode: { type: String, required: true },
```

```
    isDefault: { type: Boolean, default: false },
    location: {
      type: { type: String, enum: ['Point'] },
      coordinates: [Number] // [longitude, latitude]
    }
  }],
  isVerified: { type: Boolean, default: false },
  whatsappOptIn: { type: Boolean, default: true },
  loyaltyPoints: { type: Number, default: 0 },
  lastLogin: Date,
  createdAt: { type: Date, default: Date.now },
  updatedAt: { type: Date, default: Date.now }
}

// Indexes
UserSchema.index({ email: 1 });
UserSchema.index({ phone: 1 });
UserSchema.index({ 'addresses.location': '2dsphere' });
```

Orders Collection

```
{
  _id: ObjectId,
  orderNumber: { type: String, unique: true, required: true },
  customer: { type: ObjectId, ref: 'User', required: true },
  status: {
    type: String,
    enum: [
      'pending', 'confirmed', 'pickup_scheduled',
      'picked_up', 'processing', 'ready',
      'out_for_delivery', 'delivered', 'cancelled'
    ],
    default: 'pending'
  },
  items: [{
    category: {
      type: String,
      enum: ['clothing', 'bedding', 'accessories', 'others']
    },
    itemType: {
      type: String,
      enum: ['shirt', 'pant', 't_shirt', 'jeans', 'jacket',
             'bedsheet', 'pillow_cover', 'towel', 'suit',
             'saree', 'kurta', 'shorts', 'others']
```

```
    },
    quantity: { type: Number, required: true, min: 1 },
    service: {
      type: String,
      enum: ['washing', 'dry_cleaning', 'ironing', 'washing_ironing']
    },
    pricePerUnit: { type: Number, required: true },
    totalPrice: { type: Number, required: true }
  }],
  pickup: {
    date: { type: Date, required: true },
    timeSlot: { type: String, required: true },
    address: {
      street: String,
      city: String,
      pincode: String
    },
    status: {
      type: String,
      enum: ['scheduled', 'in_progress', 'completed', 'failed'],
      default: 'scheduled'
    },
    completedAt: Date,
    staff: { type: ObjectId, ref: 'User' }
  },
  delivery: {
    date: { type: Date, required: true },
    timeSlot: { type: String, required: true },
    address: {
      street: String,
      city: String,
      pincode: String
    },
    status: {
      type: String,
      enum: ['scheduled', 'in_progress', 'completed', 'failed'],
      default: 'scheduled'
    },
    completedAt: Date,
    staff: { type: ObjectId, ref: 'User' }
  },
  pricing: {
    subtotal: { type: Number, required: true },
    tax: { type: Number, default: 0 },
    taxRate: { type: Number, default: 18 }, // GST 18%
```

```
    discount: { type: Number, default: 0 },
    discountType: { type: String, enum: ['percentage', 'fixed'] },
    totalAmount: { type: Number, required: true },
    weightCharges: { type: Number, default: 0 },
    totalWeight: { type: Number, default: 0 }, // in kg
    currency: { type: String, default: 'INR' }
  },
  payment: {
    method: {
      type: String,
      enum: ['cash', 'online', 'wallet'],
      default: 'cash'
    },
    status: {
      type: String,
      enum: ['pending', 'completed', 'failed', 'refunded'],
      default: 'pending'
    },
    transactionId: String,
    razorpayOrderId: String,
    razorpayPaymentId: String,
    paidAt: Date
  },
  qrCode: { type: String },
  whatsappNotifications: [{
    type: {
      type: String,
      enum: ['order_confirmation', 'pickup_scheduled',
             'pickup_confirmed', 'processing', 'ready',
             'delivery_confirmed', 'payment_received']
    },
    messageId: String,
    sentAt: { type: Date, default: Date.now },
    status: {
      type: String,
      enum: ['sent', 'delivered', 'read', 'failed'],
      default: 'sent'
    }
  }],
  cancellationReason: String,
  notes: String,
  createdAt: { type: Date, default: Date.now },
  updatedAt: { type: Date, default: Date.now }
}
```

```
// Indexes
OrderSchema.index({ orderNumber: 1 });
OrderSchema.index({ customer: 1, createdAt: -1 });
OrderSchema.index({ status: 1 });
OrderSchema.index({ 'pickup.date': 1 });
OrderSchema.index({ createdAt: -1 });
```

3.3 API Architecture

RESTful API Endpoints

```
# Authentication APIs
POST   /api/auth/register              # Register new user
POST   /api/auth/login                 # Login with email/phone + password
POST   /api/auth/send-otp              # Send OTP via WhatsApp
POST   /api/auth/verify-otp            # Verify OTP
POST   /api/auth/refresh-token         # Refresh JWT token
POST   /api/auth/forgot-password       # Password reset request
PUT    /api/auth/reset-password        # Reset password with token

# User APIs (Protected)
GET    /api/users/profile              # Get user profile
PUT    /api/users/profile              # Update user profile
GET    /api/users/addresses            # Get saved addresses
POST   /api/users/addresses            # Add new address
PUT    /api/users/addresses/:id        # Update address
DELETE /api/users/addresses/:id        # Delete address

# Order APIs (Protected - Customer)
POST   /api/orders                     # Create new order
GET    /api/orders                     # Get user's orders (with filters)
GET    /api/orders/:id                 # Get order details
PUT    /api/orders/:id/cancel          # Cancel order
POST   /api/orders/:id/reorder         # Reorder from history
GET    /api/orders/:id/qr              # Get order QR code

# Pricing APIs (Public)
GET    /api/pricing                    # Get all pricing
GET    /api/pricing/:category          # Get pricing by category
GET    /api/pricing/calculate          # Calculate price based on items

# Admin APIs (Protected - Admin)
GET    /api/admin/dashboard            # Dashboard statistics
```

```
GET    /api/admin/orders               # Get all orders (with filters)
PUT    /api/admin/orders/:id/status    # Update order status
POST   /api/admin/orders/:id/pickup    # Confirm pickup
POST   /api/admin/orders/:id/delivery  # Confirm delivery
POST   /api/admin/orders/:id/whatsapp  # Send WhatsApp update
PUT    /api/admin/pricing              # Update pricing
GET    /api/admin/analytics            # Get analytics data
GET    /api/admin/customers            # Get all customers
GET    /api/admin/reports              # Generate reports
```

API Response Format

```
// Success Response
{
  "success": true,
  "data": { ... },
  "message": "Operation successful",
  "timestamp": "2026-07-07T10:30:00.000Z"
}

// Error Response
{
  "success": false,
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "Invalid input data",
    "details": [
      {
        "field": "phone",
        "message": "Phone number is required"
      }
    ]
  },
  "timestamp": "2026-07-07T10:30:00.000Z"
}

// Paginated Response
{
  "success": true,
  "data": [ ... ],
  "pagination": {
    "currentPage": 1,
    "totalPages": 5,
    "totalItems": 50,
```

```
    "itemsPerPage": 10,
    "hasNextPage": true,
    "hasPrevPage": false
```